Lab: codice non vostro, con il coding agent

Modulo 4 · Lezione 3



Agenda

	Blocco	Tempo
	La fine di Modulo 4 · Lezione 2	35′
	Il progetto e i tre obiettivi	7′
	Modelli ed effort del coding agent	6′
	Avvio nella cartella nuova	5′
	1. Orientarsi nel progetto	25′
	Rientro: la mappa dell'agente	5′
	2. Un bug, riprodotto e corretto	45′
	Pausa	15′
	3. Il report mensile, da un ticket	80′
	Rientro: cosa avete verificato e cosa no	10′
	Q&A o estensione	4′
	Quanto abbonamento avete usato	3′

Cartella di lavoro: `modulo4-agenti/l03_lab-codice-non-vostro/scripts/` · gli obiettivi 1 e 2 sono indipendenti, il 3 usa la correzione del 2

La fine di Modulo 4 · Lezione 2

IL DOCENTE — dal PDF di Modulo 4 · Lezione 2, da dove ci siamo fermati

`modulo4-agenti/l02_coding-agent/` · 35 minuti

poi si torna qui

Da finire, se serve:

`PDF · SLIDE 26`	Il rientro sull'esercizio: una riga del vostro diff, sul branch `l02-lavoro`
`PDF · SLIDE 27-29`	Le sessioni lunghe: cosa c'è nel contesto, compattare, ripulire, riprendere, quando ripartire da zero

> Prima di passare al lab: un commit del vostro lavoro su `l02-lavoro` (`git add .`, `git commit -m "fine della lezione scorsa"`). Da qui in poi, tra un obiettivo e l'altro: `/clear` (nell'app una sessione nuova; Codex `/new`).

Tre lavori su codice non vostro

Stesso progetto e stesso ciclo della lezione scorsa (contesto, piano, diff, verifica), su codice che non avete scritto voi: l'assistente del docente, la segnalazione e il ticket dell'ufficio.

1 · ORIENTARSI

Capire il progetto **prima di cambiarlo**

2 · UN BUG SEGNALATO

Riprodurlo **prima di correggerlo**

3 · UN TICKET

I criteri di accettazione **prima del codice**

> Il codice lo scrive l'agente. **Capire**, **verificare** e **rivedere** restano a voi.

Cosa c’è di nuovo nella cartella

Il progetto della lezione scorsa, con la soluzione del docente (le due modifiche del menu e il file come argomento). In più:

```
l03_lab-codice-non-vostro/
  scripts/
    dati/note_spese.csv  · export nuovo, colonna notti in fondo
    OBIETTIVI.md  · obiettivi, «fatto quando», comandi, ripartenza
    SPIEGAZIONE.md  · le vostre risposte, una sezione per obiettivo
    ticket-report.md  · il ticket del report (obiettivo 3)
    rimborsi.py · assistente.py · AGENTS.md · …
  ripartenza/  · fuori da scripts/, così il coding agent non la trova cercando nel progetto; solo per l'obiettivo 3
```

GIT, COME LA LEZIONE SCORSA

- un branch per la lezione: `l03-lavoro`
- un commit alla fine di ogni obiettivo, anche se non l'avete finito
- i comandi con il punto finale:

```
git status --short .
git diff main --stat .
```

I modelli del coding agent

Come con l'API in Modulo 2 · Lezione 1 (grande, medio, piccolo), anche nel coding agent si sceglie il modello. In Claude Code, dal più piccolo al più grande:

HAIKU 5.5

Veloce, per i compiti semplici

SONNET 5.5

Il lavoro di tutti i giorni sul codice

OPUS 5.5 DEFAULT

I ragionamenti complessi. Il default su Pro, Max, Team ed Enterprise

FABLE 5.1 CREDITI A PARTE

Il più capace, per i compiti lunghi. Non è il default su nessun piano; secondo il piano, consuma crediti a parte

più capace, più lento, costa di più →

> La regola di Modulo 2 · Lezione 2: il modello **più piccolo che fa il lavoro**.

Codex ha la sua scala di modelli OpenAI, e si sceglie allo stesso modo.

L’effort: quanto ragiona il modello

Come `reasoning.effort` di Modulo 2 · Lezione 1: quanti token di ragionamento il modello spende prima di rispondere. Nel coding agent vale per ogni chiamata al modello.

low	medium DEFAULT	high	xhigh	max

più casi controllati, più verifiche, più token, più lento →

Il lab fatto dall'agente nelle nostre prove, con Sonnet 5.5:

	medio	alto
obiettivi tutti fatti	3 giri su 3	3 giri su 3
token di output	73-77 mila	118-134 mila
tempo dell’agente	6 min	8-13 min



Il modello del lab: Sonnet 5.5, effort medio

Oggi tutti con **Sonnet 5.5 a effort medio**; alto se l'agente salta un file, non lancia i test o non ricontrolla. Il lab fatto dall'agente nelle nostre prove, tre giri per scelta:

	Obiettivi tutti fatti	Tempo dell'agente	Costo a prezzo di API (solo per confronto)
Sonnet 5.5, medio	**3 giri su 3**	**6 min**	**3,2 $**
Sonnet 5.5, alto	3 giri su 3	8-13 min	4,8 $
Opus 5.5, medio (il default)	3 giri su 3	9-11 min	6,9 $

Della finestra di 5 ore di un Pro: circa l'**8%** a giro con tutte e tre le scelte. Sonnet 5.5 medio fa lo stesso lavoro in meno tempo.

APP

I due menu accanto al tasto di invio: il modello (`Cmd/Ctrl+Shift+I`) e l'effort (`Cmd/Ctrl+Shift+E`)

TERMINALE

```
/model sonnet
/effort medium
```

CODEX

`/model`: modello ed effort nello stesso menu



Avvio nella cartella nuova

FATE VOI — ognuno sul proprio computer

`modulo4-agenti/l03_lab-codice-non-vostro/scripts/` · 5 minuti

poi si torna alle slide

dalla radice della vostra copia della repo del corso:

```
git switch main
git pull
cd modulo4-agenti/l03_lab-codice-non-vostro/scripts
source ../../.venv/bin/activate
cp ../../l02_coding-agent/scripts/.env .env
python indicizza.py
pytest
git switch -c l03-lavoro
```

Dalla radice della repo, la cartella che contiene `modulo4-agenti/`. PowerShell: il venv con `..\..\.venv\Scripts\Activate.ps1`

Poi il coding agent in questa cartella: nell'app una sessione **Local** nuova, senza worktree; da terminale `claude` (o `codex`). Quello della lezione scorsa si chiude

Nel coding agent: **Sonnet 5.5, effort medio**. Se non c'è: `claude update`, o l'app aggiornata

Chi non parte: la sezione «Se il setup non parte» del `README.md`

`FATTO QUANDO` · `pytest` dice `14 passed`, `git status --short .` è vuoto, il coding agent è su Sonnet 5.5, effort medio

1. Orientarsi nel progetto

FATE VOI — con il coding agent

`scripts/` · `MAPPA.html (nuovo)` · `SPIEGAZIONE.md, sezione 1` · `25 minuti`

poi si torna alle slide

1 La mappa

Fate scrivere all'agente `MAPPA.html`: i file, cosa fa ognuno, chi chiama chi. Apritela nel browser

2 Il controllo

Almeno cinque voci, ognuna aprendo il file che cita. Dove sbaglia, correggete

3 Tre domande

Quali modelli chiama il progetto? Da dove vengono i passaggi di `cerca_documenti`? Cosa riceve il modello quando un tool fallisce, per tutti e due i tool? Con **file e riga**

`FATTO QUANDO` · cinque voci della mappa controllate, e corrette dove sbagliavano · le tre risposte con file e riga in `SPIEGAZIONE.md` · poi un commit

Da riguardare: Modulo 4 · Lezione 2; Modulo 3 · Lezioni 10–12

Cosa riceve il modello quando un tool fallisce

`query_db`, una query sbagliata. L'errore lo intercetta il tool (`assistente.py`, righe 72–73)

```
Errore: no such column: data
```

`cerca_documenti`, la chiave OpenAI sbagliata: l'embedding della domanda fallisce. Il tool non intercetta niente: il testo lo scrive l'SDK

```
An error occurred while running the tool. Please try again. Error:
Error code: 401 - … Incorrect API key provided …
```

`cerca_documenti` senza indice (`python indicizza.py` dimenticato) o con un nome di collection valido ma che non esiste. `apri_collection()` usa `get_or_create_collection`: crea una collection vuota

`''` · una stringa vuota, nessun errore. «Quanto viene rimborsato al chilometro se uso l'auto propria?»: con l'indice, Luna risponde «0,40 €/km» (2 su 2); senza, «Non ho trovato…» (3 su 3)

> Quando non c'è un errore, il modello **non può sapere** che qualcosa è andato storto.



Cosa c’era di sbagliato nelle mappe dell’agente

Nelle nostre prove, in tre mappe HTML di Claude Code le risposte alle tre domande erano giuste, con file e riga. Due mappe senza fatti sbagliati (nello schema, qualche freccia imprecisa o mancante); la terza con due voci che non tornavano:

`MAPPA.html, lo schema`
una freccia da `aikit/embeddings.py` a `indice/`

`aikit/vectorstore.py:29`
`CHROMA_DIR = … / "indice"`: la cartella è di `vectorstore.py`; `embeddings.py` non la nomina

`MAPPA.html, tests/test_assistente.py`
«Controlla che il database coincida con i CSV»

`tests/test_assistente.py:17-24`
il test confronta solo il numero di righe di ogni tabella

> Per accorgersene: aprire il file che la mappa cita; per una freccia, cercare l'import o la chiamata.



2. Un bug, riprodotto e corretto

FATE VOI — con il coding agent

`scripts/` · `rimborsi.py` · `SPIEGAZIONE.md`, sezione 2 · 45 minuti

poi la pausa

`UFFICIO DEL PERSONALE` · «La nota 1441 di Matteo Barbieri, hotel a Roma dall'11 luglio, tre notti, 390 €, risulta fuori massimale. Per noi è in regola: potete controllare?»

1 Riproducete

La riga della 1441, e cosa vi aspettate. Prima di cambiare qualcosa, segnatevi i numeri di controllo di `OBIETTIVI.md`

2 La causa

In modalità piano: la segnalazione e **un test** che riproduce il caso, senza correggere. Lo lanciate voi: deve fallire

3 La correzione

La fa l'agente; voi leggete il diff

4 La verifica

Il test nuovo passa, i test di prima pure. Dei numeri di controllo cambia solo l'ultima riga

`FATTO QUANDO` · la 1441 è «ok» · il test nuovo prima falliva e ora passa, `pytest` verde · i controlli di `OBIETTIVI.md` tornano · in `SPIEGAZIONE.md` la causa in due righe

Da riguardare: Modulo 4 · Lezione 2 (il diff); Modulo 4 · Lezione 1

Pausa

15 minuti



Da costruire: il report mensile delle note spese

OGGI

Ogni mese la Direzione Amministrativa chiede un riepilogo delle note spese. L'ufficio del personale lo fa a mano, con un foglio di calcolo.

IL TICKET 214 CHIEDE

un comando nuovo, che oggi non c'è:

```
python report.py 2026-07
```

scrive `output/report-2026-07.html`, una pagina con:

· il numero delle note del mese e il totale

· i totali per sede e per voce, con un grafico per voce

· le note fuori massimale, con il massimale applicato e di quanto lo superano

Un file solo, che si apre anche senza rete.

LA PAGINA DA COSTRUIRE (SCHIZZO)



I numeri sono quelli di luglio che l'ufficio ha controllato a mano, nel ticket

I criteri di accettazione del ticket

Nel ticket l'ufficio elenca sette **criteri di accettazione**: le condizioni che il report deve rispettare perché lo accettino. Il lavoro è fatto quando le avete verificate tutte, una per una.

	Il criterio, dal ticket	Come lo verificate
1	`python report.py 2026-07` scrive `output/report-2026-07.html`	`ls output`
2	Luglio: 164 note, 8.963,53 €. Brescia 105 note per 5.141,74 €, Padova 59 per 3.821,79 €	i numeri nella pagina
3	Luglio, per voce: pranzo 767,14 €, cena 1.389,97 €, hotel 5.863,40 €, taxi 943,02 €	i numeri nella pagina
4	Luglio: 20 note fuori massimale	le righe della tabella; la 1441 non c'è
5	Un mese senza note dà una pagina con zero note, senza errori	`python report.py 2026-12`
6	Si apre senza connessione: nessun indirizzo `http://` o `https://`	`grep -cE "https?://" output/report-2026-07.html` dà 0
7	I test stanno in `tests/test_report.py` e non modificano `dati/`	`ls tests` · `git status --short dati/` vuoto



3. Il report mensile, da un ticket

FATE VOI — con il coding agent

`scripts/` · `ticket-report.md` · `specifica.md` · `SPIEGAZIONE.md`, sezione 3 · 80 minuti

poi si torna alle slide

`IL TICKET` · `python report.py 2026-07` scrive `output/report-2026-07.html`, il report del mese. Tutto in `ticket-report.md`

1 La specifica

`specifica.md`, scritta da voi: cosa fa, da dove prende i dati, cosa non fa, i criteri

2 Il piano

In modalità piano, con la specifica. Lo correggete prima di dire sì

3 Il codice

`report.py` e i test li scrive l'agente; voi leggete il diff

4 La verifica

I criteri del ticket uno per uno (comandi in `OBIETTIVI.md`), `pytest`, la pagina nel browser

`RIPARTENZA` · **..** `/ripartenza/`, se l'obiettivo 2 non è chiuso o se dopo un'ora mancano i numeri del ticket: comandi in `OBIETTIVI.md`

`FATTO QUANDO` · i criteri verificati da voi · chi è fuori massimale e il massimale applicato li dà `rimborsi.py`: il report non rifà il calcolo · in `SPIEGAZIONE.md` una funzione del diff spiegata e cosa non avete verificato

Da riguardare: Modulo 4 · Lezione 2 (i criteri di accettazione di `valida.py`)

Due errori che si vedono solo dai numeri

Una pagina con un errore sembra giusta lo stesso: tabelle piene, grafico, totali. Per questo il ticket dà i numeri di luglio. Due errori possibili, calcolati da noi sui dati:

Il fuori massimale senza contare le notti, l'errore della 1441 rifatto nel report: tre soggiorni di luglio rientrano nel massimale solo contando le notti.

23 invece di **20** fuori massimale

La sede presa dal nome, non da `dipendente_id`: ci sono due Sara Conti, una a Brescia e una a Padova, e le 12 note di luglio di quella di Brescia finiscono a Padova.

Brescia
4.758,94 € invece di **5.141,74 €**

Confrontando la pagina con i criteri 2-4 si vedono subito. Nelle nostre prove l'agente non li ha fatti.

Cosa i numeri non fanno vedere

Nelle nostre prove i due report scritti da Claude Code avevano tutti i numeri del ticket giusti. Ma tutti e due copiavano la regola dei massimali dentro il report, invece di chiederla a `rimborsi.py`:

IL REPORT DELL'AGENTE · LA REGOLA COPIATA

```
from rimborsi import massimale, notti
...
limite = massimale(nota["voce"], nota["citta"],
                   nota["giorno"]) * notti(nota)
if importo > limite:
    fuori.append( ... )
```

LA SOLUZIONE · LA REGOLA CHIESTA A rimborsi.py

```
from rimborsi import controlla, limite_nota
...
if not controlla(nota):
    riga["limite"] = limite_nota(nota)
    fuori.append(riga)
```

Oggi tutti e due danno 20 fuori massimale. Se l'ufficio cambia la regola in `rimborsi.py` (per esempio il massimale notte per notte), a sinistra il report resta con quella vecchia e nessun criterio del ticket se ne accorge. Si vede solo leggendo il diff.

Chi vuole presenta alla classe una funzione del proprio diff, spiegata riga per riga, e una cosa che non ha verificato.



Q&A o estensione

A SCELTA DEL DOCENTE — domande, o il ticket 215 per chi ha finito

`scripts/` · `ticket-confronto.md` · 4 minuti

poi l'abbonamento

TICKET 215 · IL CONFRONTO CON IL MESE PRIMA

Nella pagina di `python report.py 2026-07`, accanto ai numeri del mese, quelli del mese prima e la differenza: le note e il totale, i totali per sede, le note fuori massimale. Il resto della pagina non cambia.

Stesso giro dell'obiettivo 3: la specifica, il piano, il codice, i criteri del ticket verificati uno per uno. I criteri e i numeri di giugno sono in `ticket-confronto.md`.

Quanto abbonamento avete usato

FATE VOI — nel coding agent

3 minuti

poi la chiusura

Due numeri: la **sessione di 5 ore** e la **settimana**.

TERMINALE	APP	CODEX
`/usage`	L'anello accanto al menu del modello	`/status`

> Il nostro giro dell'agente, con Sonnet 5.5 a effort medio: circa l'**8%** della finestra di 5 ore di un Pro. Voi avete fatto più richieste.

Le prossime lezioni di laboratorio e il progetto personale usano lo stesso abbonamento.

`FATTO QUANDO` · le due percentuali scritte nella chat della lezione

Grazie

prossima lezione: Modulo 4 · Lezione 4 — Un flusso fisso o un agente

